

Road Damage Detection and Severity Estimation with Transfer Learning

Computer Vision Group Project — IE School of Science & Technology (Master in Business Analytics & Big Data)

Reader note: all numbers in this report are the **actual results** produced by the executed notebooks (`01_EDA` , `02_Training` , `03_Inference_Severity` , `04_Demo_App`) on a Google Colab Tesla T4. They are reproducible from the seeded pipeline (`seed=42`).

1. Introduction

Roads degrade continuously through traffic loading, weather and ageing. Potholes and cracks are the most common and most consequential surface defects: they damage vehicles, endanger cyclists and motorcyclists, and — if left untreated — accelerate structural failure of the pavement. Manual road surveys are slow, expensive and subjective. Automating the *detection* and *severity triage* of road damage from imagery lets road authorities prioritise repairs where they matter most.

This project builds a complete computer-vision pipeline that (a) **detects** four categories of road damage with a transfer-learned object detector and (b) assigns an interpretable **severity score** to every detected defect, plus an overall **road-condition index** per image. A live dashboard demonstrates the system on previously unseen images.

2. Problem definition

Given a road image, the system must:

1. **Localise and classify** each instance of road damage into one of four classes (object detection with bounding boxes).
2. **Estimate the severity** of each detected defect (Low / Medium / High and a continuous 0–1 score).
3. **Summarise** the overall condition of the road surface into a single, actionable index.

Detection is a supervised object-detection task evaluated with mean Average Precision (mAP). Severity estimation is an *unsupervised, rule-based* post-processing step, because the dataset contains no severity labels (§5, §11).

3. Motivation

- **Safety:** potholes are a direct hazard; prioritising them saves lives and vehicle-damage claims.
- **Cost:** early detection of cracking (before it becomes a pothole) makes repairs an order of magnitude cheaper.
- **Scalability:** dashcams and drones already produce road imagery at scale; an automatic detector turns that raw footage into a maintenance work-list.

- **Course fit:** the task requires **object detection** (mandatory) and benefits from **transfer learning** (a small, specialised dataset on top of a large pretrained backbone) — exactly the techniques the module targets.

4. Dataset description

We use the **RDD2022** (Road Damage Dataset 2022) **China-Drone** subset, a publicly available road-damage dataset released through CRDDC'2022 under CC BY 4.0. For this project we prepared a 300-image sample from the China-Drone subset to keep training and live-demo execution practical in Google Colab.

Property	Value
Images	300 JPEG, 512×512 px , top-down drone view
Total annotated defects	514 boxes (avg 1.71 per image; 0 background images)
Annotation format	Pascal VOC XML (one file per image)
Classes	D00 longitudinal crack, D10 transverse crack, D20 alligator crack, D40 pothole
Excluded labels	Repair , Block crack (not part of the task)

Class distribution (boxes), measured in 01_EDA :

Class	Name	Boxes	Share
D00	Longitudinal crack	179	34.8 %
D10	Transverse crack	173	33.7 %
D20	Alligator crack	84	16.3 %
D40	Pothole	78	15.2 %

Key characteristics that shaped the modelling:

- **Class imbalance:** linear cracks (**D00** + **D10** = 352 boxes, ~68 %) dominate; **alligator cracking and potholes are roughly half as frequent** (84 and 78 boxes). This is the single most important fact for reading the per-class metrics in §12.
- **Few defects per image** (1.71 on average) and **no background-only images**.
- **Small, often elongated boxes:** crack boxes have extreme aspect ratios, making this primarily a **small-object detection** problem.

5. Data preprocessing

1. **Parsing.** Each VOC XML is parsed into typed dataclasses in the embedded notebook helper modules; out-of-vocabulary classes are dropped and coordinates are clamped to the image bounds to remove degenerate boxes.
2. **VOC → YOLO conversion.** Each box (xmin, ymin, xmax, ymax) is converted to normalised **class cx cy w h** , the format Ultralytics expects.

3. **Reproducible stratified split (70/20/10)**. With so few potholes/alligator boxes, a naive random split risks leaving a class absent from a split. We order images by their *rarest contained class* and deal them into train/val/test by target share. The resulting split (seeded, `seed=42`):

Split	Images	Boxes	D00	D10	D20	D40
train	210	361	134	113	60	54
val	60	105	29	43	16	17
test	30	48	16	17	8	7

Every class is present in every split, so the test metrics are meaningful for all four classes.

4. `data.yaml` is generated automatically (4 classes, the three split paths).
5. **No manual re-annotation was required** — the dataset ships already annotated, so we go straight from Pascal VOC to transfer learning.

6. Model architecture

We use **YOLO26** (Ultralytics 8.4.78), a modern single-stage convolutional detector that predicts boxes and class scores in one forward pass: a convolutional **backbone** for feature extraction, a **neck** (feature-pyramid / path-aggregation) fusing multi-scale features, and a **detection head** emitting boxes, objectness and class logits. The detection loss combines **CIOU** box regression, **BCE** classification and **distribution focal loss (DFL)**.

The training helper module embedded in `02_Training.ipynb` resolves a YOLO26 COCO checkpoint and falls back to YOLOv8 of the same scale if needed; in our runs the genuine **YOLO26** weights (`yolo26{n,s,m}.pt`) were downloaded and used. We trained **three scales** — **nano, small, medium** — for a real speed–accuracy comparison.

7. Why transfer learning

The dataset has only **210 training images**, far too few to train a detector from scratch. We initialise from weights **pretrained on COCO** (80 everyday-object classes, >100k images). The backbone has already learned generic visual primitives — edges, textures, shapes — that transfer directly to road imagery; fine-tuning adapts the higher layers and the detection head to the four road-damage classes. This mirrors the course material on transfer learning (reuse a pretrained feature extractor; retrain the task-specific top), applied to detection rather than classification. The payoff is fast convergence and far less overfitting than random initialisation would allow on 210 images.

8. Hyperparameters

Centralised in the embedded `config.TrainConfig` module and printed in `02_Training`:

Hyper-parameter	Value
Pretrained weights	YOLO26 COCO (<code>yolo26{n,s,m}.pt</code>)
Scales trained	n, s, m
Optimizer	AdamW
Initial learning rate <code>lr0</code>	1e-3
Final LR factor <code>lrf</code>	1e-2
LR scheduler	Cosine
Weight decay	5e-4
Warmup epochs	3
Epochs (max)	100 (early stopping, patience 20)
Image size	512 (native)
Batch size	16
Mixed precision (AMP)	Enabled
Augmentation	Mosaic 1.0 (closed last 10 epochs), HSV (0.015/0.5/0.4), horizontal flip 0.5, scale 0.5, translate 0.1
Loss	CIoU (box) + BCE (cls) + DFL
Seed	42
Hardware / stack	Colab Tesla T4 (15 GB) , Ultralytics 8.4.78, PyTorch 2.11.0+cu128, Python 3.12

Augmentation rationale. Heavy mosaic multiplies the effective number of small objects per batch — valuable given 210 images and tiny defects. HSV jitter handles lighting/asphalt-colour variation; vertical flip is disabled to preserve capture geometry; mosaic is switched off for the final 10 epochs so the model fine-tunes on natural, un-stitched images.

9. Training process

Each scale runs up to 100 epochs with cosine LR decay and early stopping on validation mAP. The best checkpoint (`best.pt`), `results.csv` , `results.png` and the confusion matrix are saved to Drive under `models/` . The training curves (`figures/train_curves_{n,s,m}.png`) show the validation losses decreasing and the validation mAP rising and then plateauing — i.e. the patience and augmentation are well chosen and there is no late-epoch divergence.

10. Evaluation metrics

Evaluation is on the **held-out test split** (30 images, 48 boxes — never seen in training or validation). We report **mAP50**, **mAP50-95**, **precision**, **recall**, **F1**, **per-class AP**, **parameter count**, **inference speed/FPS**, and the **confusion matrix**.

11. Severity estimation (project novelty)

RDD2022 has **no severity ground truth**, so severity is an **engineered, transparent index** rather than a learned label — every term is explainable:

```
severity = clip( base[class] + geometry_term , 0, 1 ) → Low / Medium / High
```

- **base[class]** encodes structural meaning: pothole (0.60) and alligator cracking (0.55) start higher than transverse (0.30) and longitudinal (0.25) cracks.
- **geometry_term** scales with footprint: for potholes/alligator it grows with the **square-root of the area fraction**; for linear cracks with **relative length and thickness**.
- Thresholds (0.34, 0.67) map the score to **Low / Medium / High**.
- The image-level **condition index** is dominated by the worst single defect and nudged up when many coexist: $\text{index} = \text{max_severity} + 0.05 \cdot (\text{n_defects} - 1)$ (capped at 1.0).

Face-validity check (measured on the test set, 03). Mean severity score by class came out exactly as the design intends — structural defects score *High*, hairline cracks score *Low/Medium*:

Class	Mean severity	Bucket
D40 Pothole	0.723	High
D20 Alligator crack	0.696	High
D00 Longitudinal crack	0.414	Medium
D10 Transverse crack	0.404	Medium

At image level, the 30 test images were triaged as **None** ×11, **Medium** ×13, **High** ×6 — a usable maintenance work-list ordering. A proper validation would correlate the index against expert ratings or repair-cost data (§16).

12. Experimental results

Model-scale comparison on the test split (`outputs/model_comparison.csv`):

Model	Params (M)	mAP50	mAP50-95	Precision	Recall	F1	Speed (ms)	FPS
YOLO26n	2.51	0.386	0.233	0.358	0.365	0.361	15.6	63.9
YOLO26s	9.95	0.540	0.268	0.506	0.538	0.521	16.1	62.1
YOLO26m	21.78	0.538	0.256	0.376	0.584	0.457	20.3	49.3

Best model: YOLO26s — highest mAP50, mAP50-95 and F1, at essentially the same speed as the nano model. A clear, classic small-data result: **the mid-size model beats the large one** (YOLO26m has ~2× the parameters but slightly *lower* mAP and much lower precision — it begins to **overfit** 210 images). YOLO26m does, however, have the **highest recall (0.584)**.

Per-class AP — YOLO26s (`02_Training`):

Class	AP50	AP50-95	Precision	Recall
D40 Pothole	0.875	0.474	0.537	1.000
D10 Transverse crack	0.611	0.283	0.633	0.588
D00 Longitudinal crack	0.487	0.230	0.528	0.438
D20 Alligator crack	0.186	0.083	0.326	0.125

The most important finding — and a surprise vs. our prior expectation. Despite being one of the rarest classes, **potholes are detected best** (AP50 0.875, recall 1.0): they are dark, high-contrast, compact blobs that the COCO-pretrained backbone localises easily even from few examples. The **hardest class is alligator cracking (D20)** (AP50 0.186, recall 0.125): it is a diffuse, low-contrast *texture* rather than a sharp edge, is easily confused with normal road texture and with other crack types, and has only 60 training boxes. The two linear-crack classes sit in between, with the most frequent confusion being **D00 ↔ D10** (orientation-dependent).

Deployed model for inference & demo: YOLO26m. 03 and 04 load YOLO26m deliberately, because for a **maintenance triage tool, recall matters more than precision** — missing a defect (false negative) is costlier than a spurious box a human can dismiss — and YOLO26m has the best recall (0.584). YOLO26s remains the best *overall* detector and is the model to cite for headline accuracy.

13. Error analysis

The five most informative test cases (largest |GT – predicted| box count, from 03’s failure analysis, [figures/failure_cases.png](#)):

Image	GT boxes	Predicted	Failure mode
China_Drone_000839	5	2	Missed 3 defects (false negatives)
China_Drone_000955	3	0	Missed all (false negatives)
China_Drone_000022	2	0	Missed all (false negatives)
China_Drone_000036	2	0	Missed all (false negatives)
China_Drone_000158	2	5	Over-prediction (fragmented / false positives)

Four of the five worst cases are misses, confirming the metrics: recall (~0.54–0.58) is the binding constraint, not precision. The recurring failure modes and fixes:

- Missed faint / texture-like cracks (dominant).** Most misses are low-contrast longitudinal/alligator cracks. *Fix*: higher-resolution or sliced (SAHI-style) inference; collect more such examples; class-weighted loss.
- Alligator cracking under-detected (recall 0.125).** *Fix*: treat it as a texture-segmentation problem (a **yolo-seg** head) rather than box detection; targeted augmentation.
- Fragmented / over-counted cracks (case 000158).** One physical crack reported as several boxes, or texture spawning extra boxes. *Fix*: merge collinear/overlapping boxes in post-processing.
- Crack-type confusion D00 ↔ D10.** Orientation-dependent, blurred by flips. *Fix*: disable rotational augments, or merge into one “linear crack” class if the use case allows.

5. **Background false positives** on tar seams, shadows and lane markings. *Fix*: add hard-negative backgrounds; raise the deployment confidence threshold (the demo slider makes this trade-off visible live).

14. Limitations

- **Small dataset (300 images, single source)**. 210 training images and a 30-image test split mean metrics carry real variance; the China-Drone domain may not transfer to dashcam or other-country roads.
- **No severity ground truth** — the severity index is a reasoned heuristic, not a validated measurement.
- **Scale ambiguity** — without camera calibration/depth, box size is a proxy for real-world defect size.
- **2D boxes ignore depth** — a pothole's danger depends on its depth, which a single image cannot recover.
- **Alligator cracking** remains poorly detected (recall 0.125).

15. Ethical considerations

- **Safety-critical use**: the system is a triage aid, **not** a replacement for professional inspection; the measured false-negative rate is real and must be communicated.
- **Privacy**: road imagery can incidentally capture people, vehicles and number plates; deployments should blur/avoid personal data and comply with GDPR.
- **Fairness of maintenance**: an automated work-list could entrench bias if imagery coverage is uneven across neighbourhoods; coverage should be audited.
- **Licensing**: RDD2022 is used under CC BY 4.0 with attribution.

16. Future work

- **Instance segmentation** (YOLO-seg / Mask R-CNN), especially for alligator cracking, for pixel-accurate extent and better severity-area estimates.
- **Sliced/tiled inference (SAHI)** and higher training resolution to recover faint cracks (the dominant failure mode).
- **Severity validation** against expert ratings or repair-cost records, then *learning* severity (regression) instead of hand-crafting it.
- **Class-imbalance handling**: focal-loss tuning, oversampling, or synthetic pothole/alligator augmentation.
- **Architecture comparison**: benchmark a transformer detector (e.g. RF-DETR) against YOLO for small-object robustness.
- **Geo-tagging & mapping**: attach GPS to detections to produce a maintenance heat-map.

17. Conclusion

We delivered an end-to-end road-damage system: reproducible VOC→YOLO preprocessing with a stratified split (210/60/30), **transfer-learned YOLO26** detectors at three scales, a full evaluation (the best model, **YOLO26s**, reaching **mAP50 0.540 / mAP50-95 0.268**), a transparent **severity-scoring** model whose

per-class behaviour matches its design, and an interactive **live demo** that runs on new images. The headline empirical lessons — the **mid-size model beats the larger one** on this small dataset, **potholes are the easiest** and **alligator cracking the hardest** class, and **recall is the binding constraint** — are all supported by the measured metrics and the failure analysis, and they directly motivate the future-work roadmap.

Appendix A — Reproducibility

- Code: four self-contained notebooks (`01-04`) with embedded modular helper code written to `/content/src/` at runtime for reproducibility in Colab.
- Determinism: `seed=42` for the split and training.
- Environment: `requirements.txt` ; Colab Tesla T4, Ultralytics 8.4.78.
- Data: RDD2022 China-Drone 300-image subset (CC BY 4.0).
- Generated artefacts: `outputs/model_comparison.csv` , `outputs/test_condition_summary.csv` , `figures/*.png` , `models/*/weights/best.pt` .